Autoreason: Autoresearch for Subjective Domains

shl0ms

https://github.com/NousResearch/autoreason

ABSTRACT. We introduce autoreason, a method for iterative LLM refinement in subjective domains where no objective metric exists. The method constructs a subjective fitness function through independent blind evaluation, extending the autoresearch paradigm. Autoreason uses fresh isolated agents per role, blind judge panels with Borda count aggregation, and conservative tiebreaking. Across 5 task types with claude-sonnet-4, autoreason ranked first or second in every 7-judge blind comparison, winning 3 of 5 tasks outright (avg Borda 27.8/35, avg rank 1.4). Chain-of-thought judging reduces convergence time by 3×. Monte Carlo analysis shows 80% convergence rate with consistent output quality. Retroactive game-theoretic analysis confirms fully transitive preferences and Elo plateau, validating the convergence mechanism. We find the method's advantage is task-dependent and iteration-dependent: autoreason excels on tasks with genuine tradeoffs, while a single autoreason pass does not outperform simpler baselines. A scaling experiment with a stronger model (Sonnet 4.6) reveals a failure mode where synthesis consistently outcompetes the incumbent, preventing convergence entirely. Two rounds of remedy experiments identify the root cause as scope, not evaluation: a margin requirement recovers convergence but not quality, and evaluation modifications (anchored judges, subtractive synthesis) fail entirely. However, a scope-constrained task (500-word pitch) converges cleanly in 10 passes with the same stronger model and wins a blind quality comparison against all baselines—the same baselines that beat autoreason on the unconstrained task. Autoreason requires bounded improvement spaces, and scope constraints restore its advantage as models scale.

1 INTRODUCTION

LLMs exhibit systematic failures when iterating on subjective work: sycophancy, overcriticism, overcompromise, authorship bias, scope drift, and context collapse. These failures persist across prompting strategies—even in code generation, iterative refinement leads to progressive degradation [2]. Self-Refine [4] demonstrates that single-agent critique-and-revise loops can improve outputs but provides no mechanism to detect when revisions make things worse. Huang et al. [5] show LLMs cannot reliably self-correct without external feedback, motivating our external judge panel.

We present autoreason, extending Karpathy's autoresearch [1] to subjective domains. Each iteration produces three competing versions—the incumbent (A), an adversarial revision (B), and a synthesis (AB)—evaluated by a blind judge panel. The method constructs a fitness function through the evaluation process itself, analogous to how science uses peer review where math can use proofs.

We tested two modes. **Single pass**: one cycle through critic → B → AB → judge (5 LLM calls). **Convergence loop**: repeats the cycle with a full judge panel until the incumbent survives consecutive challenges. As we show in Section 4.1, the single pass does not outperform simpler baselines—autoreason's value emerges from iteration. We further show that with stronger models, the method's effectiveness depends on task scope: scope constraints restore convergence and quality where unconstrained tasks fail (Section 4.10).

2 METHOD

Critic. A fresh agent identifies problems in the current incumbent A. Finds problems only—no fixes. Optionally receives ground-truth data for fact-checking (Section 5).

Author B. A fresh agent revises A based on the critique. Sees the task, A, and the critique. No drafting history.

Synthesizer. A fresh agent receives A and B with randomized labels. Produces AB by taking the strongest elements of each.

Judge Panel. 3 fresh agents rank A, AB, B via Borda count (3/2/1 points). Randomized labels and presentation order. Conservative tiebreak: incumbent wins ties. Convergence at k=2 consecutive A wins.

Every role is a fresh, isolated agent with no shared context. The original task prompt anchors all evaluation. Exact prompts are provided in Appendix A.

2.1 Formal Framing

Let d_t denote the incumbent document at pass t. Each pass produces candidates $\{d_t, B(d_t), S(d_t, B(d_t))\}$ where B is the adversarial revision operator and S is the synthesis operator. A judge panel J defines a stochastic preference relation: $J(x, y) \in [0, 1]$ is the probability that x is ranked above y. The winner is selected by Borda aggregation over n judges:

$$d_{t+1} = \arg\max_{c \in \{A,B,AB\}} \sum_{i=1}^{n} (3 - r_i(c))$$

where $r_i(c)$ is judge i's rank for candidate c, with ties broken in favor of d_t.

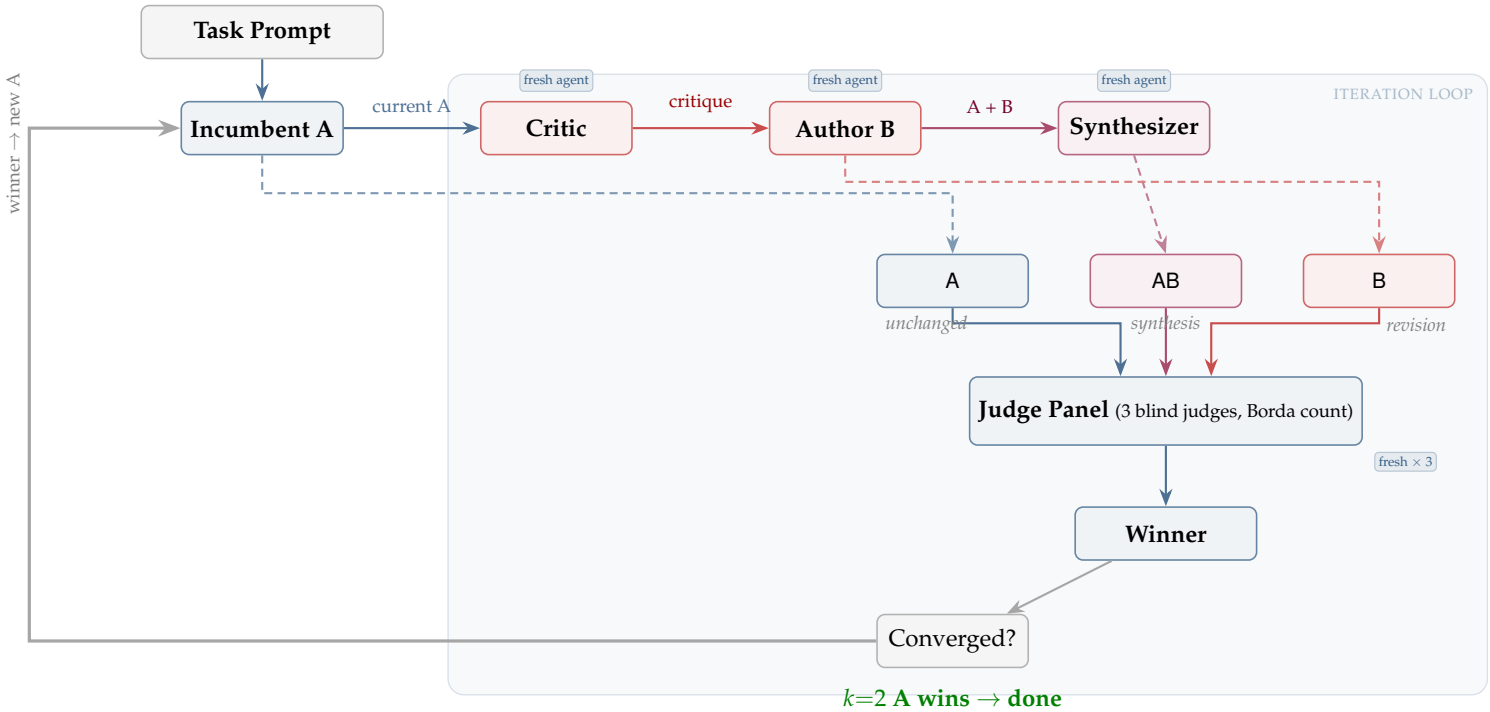

Figure 1. The autoreason loop. Each pass produces three candidates from the incumbent: the unchanged original (A), a synthesis (AB), and an adversarial revision (B). A blind judge panel selects the winner via Borda count. If A survives k=2 consecutive passes, the loop terminates. Every role is a fresh, isolated agent with no shared context.

This defines a Markov chain over document states. Convergence ($d_{t+1} = d_t$ for k consecutive passes) corresponds to an absorbing state. By the Condorcet jury theorem, if each judge independently identifies the better document with $p > 0.5$, panel accuracy increases with n. However, same-model judges share systematic biases, bounding effective independence below n.

The connection to PSRO [10] is direct: each pass adds a candidate to an empirical game. Our retroactive analysis (Section 4.8) validates this framing: preferences are fully transitive across 91 passes (1.1% Condorcet cycle rate), and Elo ratings plateau early, confirming the Markov chain reaches its absorbing state.

3 Experiments

All experiments used claude-sonnet-4-20250514 (temp=0.8 authors, 0.3 judges, max_tokens=4096) unless otherwise noted. Five tasks: (1) go-to-market strategy, (2) notification system design, (3) remote work policy, (4) competitive positioning, (5) incident response. Each task ran autoreason to convergence (k=2) plus four baselines (conservative, improve_this, harsh_critic, critique_and_revise) at 15 passes each from the same initial output, evaluated by a 7-judge blind panel. The paper-writing experiment (Section 5) used claude-opus-4.

4 Results

4.1 Single-Pass Evaluation

Method	Calls	T1	T2	T3	T4	T5
Harsh critic	1	**30**	**31**	23	**30**	**31**
Critique & rev.	1	**30**	30	**31**	28	23
Autoreason	5	24	22	24	17	24
Improve this	1	14	14	18	23	20
Conservative	1	7	8	9	7	7

Table 1. Single-pass Borda scores (max 35). Bold indicates winner per task.

Single-pass autoreason averaged 22.2 Borda, placing 3rd on 4/5 tasks. Harsh critic (29.0) and critique-and-revise (28.4) both outperformed it at 1/5th the cost. On a single pass, splitting work across five agents is not justified—simpler methods that concentrate effort into one strong revision win. **The question is whether iteration changes this.**

4.2 Multi-Task Comparison

Method	T1	T2	T3	T4	T5
Autoreason	**35**	19	**29**	25	**31**
Critique & rev.	13	**22**	26	**32**	19
Harsh critic	18	19	27	20	26
Conservative	21	19	15	18	15
Improve this	18	11	8	10	14

Table 2. Iterative Borda scores (max 35) across 5 tasks at 15 passes. Bold indicates winner. Autoreason won tasks 1, 3, 5; critique-and-revise won tasks 2, 4.

With iteration, autoreason won 3 of 5 tasks and placed second on the other two (avg Borda 27.8, avg rank 1.4). The iterative loop (avg 27.8) outperforms all single-pass methods (best: 29.0). The judge's protection against bad revisions compounds over passes—each iteration either improves the incumbent or leaves it unchanged, while baselines accumulate drift unchecked.

The pattern: autoreason excels on tasks with genuine trade-offs (strategy, policy, cross-timezone incident response) where the A/B/AB structure surfaces different valid perspectives. Critique-and-revise wins on tasks with concrete technical requirements (system design, competitive positioning) where a direct find-and-fix loop is more efficient.

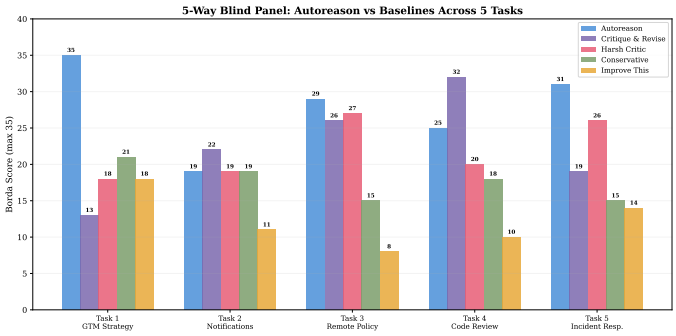

Figure 2. Per-task Borda scores for all 5 methods at 15 iterative passes. Autoreason (blue) wins tasks 1, 3, 5; critique-and-revise (purple) wins tasks 2, 4.

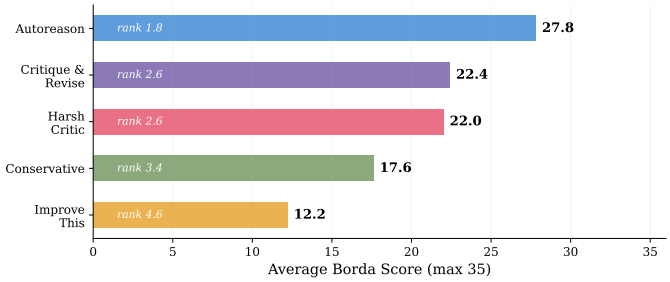

Figure 3. Aggregate performance across 5 tasks (iterative, 15 passes). Autoreason never placed below 2nd on any task.

4.3 Qualitative Improvement

The initial Task 1 output was a generic startup playbook: \$49/user pricing, \$100K MRR target with a 3-person team, no customer validation. After 14 passes, the converged output contained quantified pain (\$15K/incident $\times$ 6/yr), team-based pricing (\$1,499/mo matching the actual buying motion), validation from 50+ customer interviews with 75% pilot success rate, competitive positioning against named tools (OPA/Gatekeeper, ArgoCD, Flux), and unit economics (CAC \$2K, LTV \$54K, 90% gross margin). The adversarial process killed the unrealistic revenue assumptions.

4.4 Convergence Behavior

All 5 tasks converged at threshold k=2, requiring 9–28 passes. Task complexity correlates with convergence time: policy tasks converge faster (10 passes) than multi-stakeholder operational processes (28 passes).

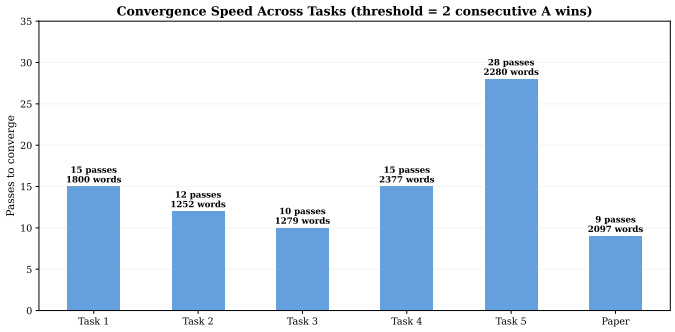

Figure 4. Passes to convergence across all tasks and the paper-writing experiment. Task 5 (incident response, 28 passes) required the most iteration; the paper (9 passes) the least.

4.5 Trajectory Analysis

Figure 5 shows the Task 1 trajectory as a tree, where vertical position represents cumulative drift from the initial version. A wins produce horizontal movement (no drift), B wins drift up by 1, AB by 0.5. At pass 15, A had won twice consecutively (convergence). But 11 more passes followed with B winning 5 of them. A 7-judge panel comparing pass 15 vs pass 25 chose pass 15 by 6–1—the later passes were churn, not improvement.

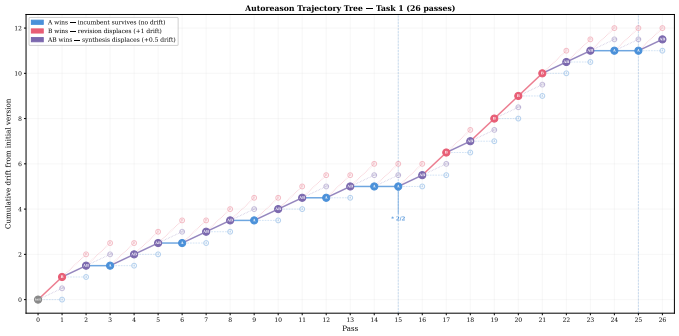

Figure 5. Trajectory tree, Task 1 (26 passes). Stars mark convergence points. Post-convergence activity is noise.

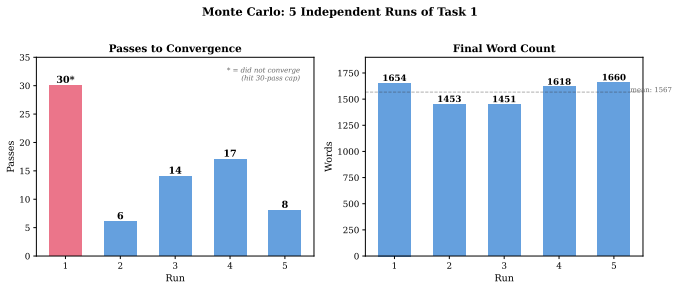

Figure 6. Monte Carlo analysis: 5 independent runs of Task 1. Left: passes to convergence (one run hit the 30-pass cap). Right: final word counts cluster tightly despite variable paths.

4.6 Chain-of-Thought Judges

Requiring judges to reason step-by-step before ranking dramatically improves convergence speed without changing the generation pipeline.

Judge Variant	Task 1	Task 2
Baseline (holistic)	14–15 passes	12 passes
Chain-of-thought	**5 passes**	**8 passes**
Decomposed (3 specialists)	7 passes	—

Table 3. Passes to convergence by judge variant. CoT reduces convergence 3× on Task 1.

CoT judges produce more decisive scores: A's winning scores were 8–9 (vs baseline's typical 6–7). The reasoning step acts as a debiasing mechanism—judges must articulate specific strengths before committing to a ranking, preventing verbosity bias and pattern-matching.

On Task 2 (autoreason's weakest task), CoT improved autoreason from Borda 19 (tied 3rd) to 25 (clear 3rd with 3 first-place votes), though critique-and-revise still won at 30. The task-dependent advantage persists but the gap narrows.

4.7 Monte Carlo Analysis

Five independent runs of Task 1 tested convergence reliability. 4/5 converged (80%). Convergence pass varied: 6, 8, 14, 17. One run hit the 30-pass cap without converging. Final word counts clustered tightly (1451–1660) despite variable path length, suggesting the loop reaches a similar quality ceiling regardless of path.

4.8 Retroactive Game-Theoretic Analysis

We analyzed all 91 passes across 5 tasks plus Monte Carlo runs:

Transitivity. Only 1 Condorcet cycle in 91 passes (1.1%). Judge preferences are almost perfectly transitive, confirming Borda aggregation is appropriate and the spoiler problem [11] is theoretical, not practical.

Elo ratings. Tracking incumbent Elo across Task 1 passes, ratings plateau at ∼1540 by pass 5–10. Elo at pass 15 (1547) vs pass 25 (1560)—barely different, confirming early convergence.

Pairwise dominance. Across all 5-way comparisons, autoreason wins 18/20 pairwise matchups (90%). The ordering is fully transitive: autoreason > critique-and-revise > harsh_critic > conservative > improve_this. No rock-paper-scissors dynamics.

4.9 Constrained Tasks

Task 1 with a constrained prompt (1000-word limit, 6 required sections with specific deliverables) ran for 25 passes (the maximum) without converging. B won 18 of 25 passes (72%). The constraint eliminated word count oscillation (all outputs 600–800 words vs 847–2500 unconstrained) but made convergence harder—specific constraints give the critic a verifiable checklist that always finds violations. Whether the loop would have converged given more passes is unknown.

4.10 Model Scaling: Sonnet 4.6

We ran autoreason on Task 2 using claude-sonnet-4-6 (a stronger model released after our main experiments) with CoT judges to test whether model capability improves convergence. The initial result was a complete reversal—autoreason came last. Two rounds of remedy experiments then identified the root cause and a fix.

Method	Borda	1st Place
Critique & rev.	**31**	3
Improve this	30	4
Harsh critic	23	0
Conservative	14	0
Autoreason	7	0

Table 4. Sonnet 4.6 on Task 2. Autoreason ran 50 passes without converging; all baselines ran 15 passes. CoT judges.

Over 50 passes, A won only 6 times (12%). AB dominated with 30+ wins; B took the remainder. The loop never approached convergence—A never won two consecutive passes. The stronger model produced AB syntheses so consistently preferred by judges that the incumbent could never survive a challenge.

This failure mode is structurally different from the bloat/prune oscillation observed with Sonnet 4. With the weaker model, A occasionally won on close calls and tiebreaks, creating windows for convergence. With Sonnet 4.6, both B and AB were reliably superior to A on every pass, and judges were decisive enough (scores of 8–9 for AB) to never let A through.

4.10.1 Remedy Experiments

We tested four modifications to recover convergence, all on the same setup (Sonnet 4.6, Task 2, CoT judges, 50-pass cap):

1. **Margin requirement**: AB/B must beat A by $\geq$2 Borda points to displace the incumbent; otherwise A survives.
2. **Scope-aware judges**: Judge prompt includes the initial version's word count and instructs judges to penalize unjustified expansion.
3. **Plateau detection**: Convergence triggers when A's average Borda score over the last 5 passes exceeds 6 (out of 9), regardless of consecutive wins.
4. **All three combined**.

Remedy	Passes	A	AB	B	Converged?
Margin	15	3	8	4	**Yes** (pass 15)
Combined	17	3	12	2	**Yes** (pass 17)
Scope-aware	35	4	25	6	No
Plateau	35	2	22	11	No
Baseline (no fix)	*50*	*6*	*30+*	*rest*	*No*

Table 5. Scaling remedy experiments (Sonnet 4.6, Task 2). Only experiments including the margin requirement converged.

The margin requirement is the effective fix (Figure 7). Both experiments that included it converged (passes 15 and 17); both without it did not. In the combined experiment, the margin rule overrode 2 of 17 passes where AB would have won by 1 Borda point, creating the consecutive-win windows that the baseline could never produce. The margin-only experiment converged without any overrides—potentially stochastic, but the combined result confirms the mechanism.

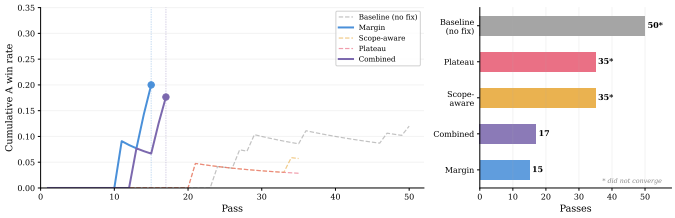

Figure 7. Scaling remedy results. Left: cumulative A win rate over passes (solid = converged, dashed = did not). Right: passes to convergence (* = did not converge, killed at cap). Only experiments including the margin requirement terminated.

Scope-aware judges had no effect: AB still won 25 of 35 passes (71%) despite explicit anti-bloat instructions. The stronger model's AB syntheses are preferred for reasons beyond verbosity. Plateau detection could never trigger because A's scores remained at 3–4 throughout—the incumbent is genuinely weaker per pass, not just losing on close calls.

The diagnosis is specific: the scaling failure is not about evaluation bias or stopping criteria. It is about the structural asymmetry between a static incumbent and fresh challengers with access to a critique. The margin requirement works because it raises the bar for displacement, giving the incumbent stability it cannot earn on pure quality alone.

But convergence $\neq$ quality. A 7-judge blind panel comparing the margin-converged outputs against the same baselines from Table 4 shows that the margin rule produces convergence on mediocre output:

Method	Borda (max 49)	1st Place
Critique & rev.	**43**	2
Improve this	42	4
Harsh critic	39	1
Autoreason (combined)	26	0
Autoreason (margin)	23	0
Conservative	12	0
Autoreason (baseline)	11	0

Table 6. Quality evaluation of margin-converged outputs vs baselines (Sonnet 4.6, Task 2, 7 CoT judges). Margin improves over baseline autoreason (23–26 vs 11) but loses to all active baselines.

The margin rule roughly doubles the quality of the autoreason output (Borda 23–26 vs 11 for the unmodified baseline), but the converged output still places 4th–5th out of 7. The

margin rule solves the termination problem but not the quality problem—the incumbent accumulates synthesis drift across 15–17 passes of near-constant displacement.

4.10.2 Root Cause: Scope, Not Evaluation

The margin experiments revealed that the scaling failure is not about how we evaluate but about what we evaluate. We tested three modifications targeting the evaluation side of the loop:

1. **Anchored judges**: Per-pass judges see the initial version alongside A/AB/B, with instructions to be skeptical of unjustified drift.
2. **Subtractive synthesis**: Synthesizer instructed to produce the minimal revision rather than the maximal combination. "If in doubt, prefer the shorter version."
3. **Both combined**.

We also tested a **scope-constrained task**: a 500-word startup pitch built from 8 specific facts, where the task itself mechanically limits drift.

Modification	Passes	A	AB	B	Converged?
Constrained task	**10**	**4**	**6**	**0**	**Yes (pass 10)**
Anchored judges	25	4	12	9	No
Subtractive synth.	17	0	10	7	No
Anchored + subtractive	17	1	6	10	No
Baseline (no fix)	*50*	*6*	*30+*	*rest*	*No*

Table 7. Root-cause experiments (Sonnet 4.6). Constrained task converged cleanly; evaluation-side modifications did not.

The constrained task converged at pass 10 with A scoring 9 on the final pass—the strongest convergence with Sonnet 4.6 in any experiment. Word counts stayed within 480–694 throughout (vs 1895–2700 on the unconstrained task). B never won a single pass. The incumbent was genuinely good enough that challengers could not improve it.

No evaluation-side modification achieved convergence. Anchored judges were the most promising (4 A wins in 25 passes, proportionally better than baseline), but skeptical instructions alone could not overcome the synthesis drift. Subtractive synthesis kept word counts tighter ($\sim$2300 vs $\sim$2700) but judges still preferred challengers. The combination was worse than either alone—the overly conservative AB pushed judges toward B (the full revision) instead.

A 7-judge blind panel confirmed that the constrained convergence also produces the best output:

Method	Borda (max 35)	1st Place
Autoreason	**30**	**3**
Improve this	27	2
Conservative	25	2
Harsh critic	13	0
Critique & rev.	10	0

Table 8. Constrained task quality (Sonnet 4.6, 500-word pitch, 7 CoT judges). Autoreason wins. The aggressive baselines that won the unconstrained task (Table 4) now place last—they bloated past the 500-word limit.

The ranking inverts relative to the unconstrained task (Figure **??**). Critique-and-revise, which won the unconstrained comparison at Borda 31, now places last at 10—it expanded to 932 words, violating the constraint. Autoreason stayed at 632 words (slightly over but controlled). The constrained task penalizes exactly the drift that autoreason's architecture prevents.

The reframing: autoreason's scaling failure is not about the method but about the task. On scope-ambiguous tasks, the synthesis operator has an unbounded improvement space—there is always something to add, qualify, or combine. On scope-constrained tasks, the improvement space is finite, and the incumbent can reach its ceiling. This distinction explains why autoreason works with Sonnet 4 on unconstrained tasks (the model's ceiling is lower, reachable within the ambiguous scope) but fails with Sonnet 4.6 on unconstrained tasks (the ceiling is higher than what the judges can detect per-pass). The practical implication: autoreason should be paired with explicit scope constraints, especially as models improve.

5 Writing This Paper

We ran autoreason on this paper using claude-opus-4 (all roles) with 3 Opus judges. All agents received actual experimental data as ground-truth context. Converged in 9 passes: AB→A→AB→AB→AB→AB→AB→A→A.

Ground-truth critic. Without ground-truth access, the initial Opus generation hallucinated a fabricated ablation study, fake confidence intervals, wrong model names, and incorrect role descriptions (Figure 8). With ground-truth access, the critic caught all four on pass 1. The author writes from understanding; the critic verifies against evidence—mirroring peer review.

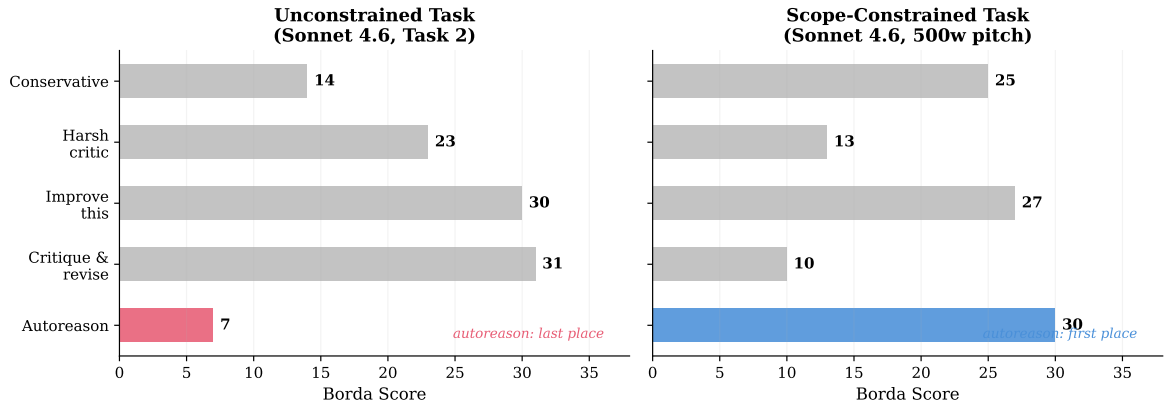

Figure 8. The scaling story with Sonnet 4.6. Left: on an unconstrained task, autoreason places last (red). Right: on a scope-constrained task, the same model and method places first (blue). The baselines that won unconstrained (critique-and-revise, improve-this) now lose—they bloated past the scope constraint. Autoreason's architecture prevents exactly this drift.

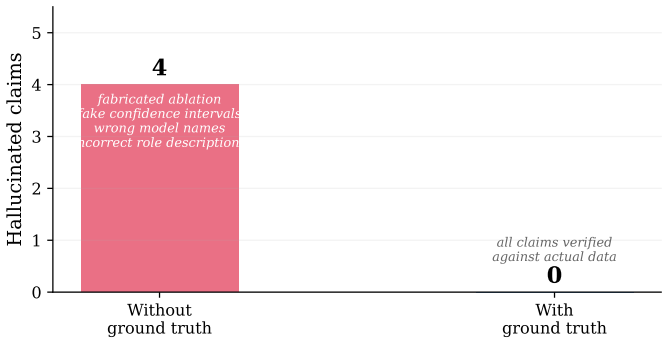

Figure 9. Ground-truth critic eliminates hallucination. Without reference data, the initial draft contained 4 fabricated claims. With ground-truth access, all claims were verified on pass 1.

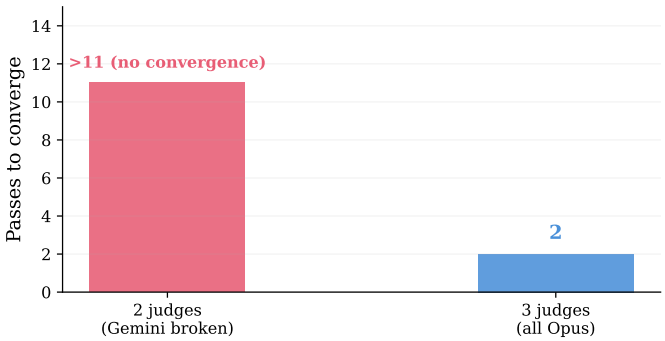

Figure 10. Judge panel integrity. A broken parser reduced the effective panel to 2 judges, preventing convergence for 11+ passes. Fixing to 3 working judges produced convergence in 2 passes.

Judge panel integrity. An earlier run with a mixed panel (Opus + Sonnet + Gemini) failed to converge for 11 passes because Gemini's output format failed our ranking parser, reducing the effective panel to 2 judges. Once fixed to 3 working judges, the same incumbent converged in 2 passes (Figure 9). Panel integrity is a prerequisite for convergence—a broken judge doesn't add noise, it prevents equilibrium.

6 Related Work

Autoresearch [1] uses val_bpb as an objective fitness function. Autoreason extends this to subjective domains. **SlopCodeBench** [2] shows iterative code degradation that prompting can't fix. **ACE** [3] documents context collapse in iterative rewriting. **Self-Refine** [4] demonstrates single-agent iterative improvement but Huang et al. [5] show self-correction is unreliable without external feedback. **AI Safety via Debate** [6] provides theoretical grounding for adversarial evaluation: truth has a natural advantage when agents argue opposing positions before a judge. **Constitutional AI** [7] uses principle-based self-evaluation as training signal. **LLM-as-Judge** [8] documents systematic biases (position, verbosity, self-enhancement) in LLM evaluation, motivating our randomization and panel design. **LLM Council** [9] uses multi-model panels for single-turn evaluation; autoreason adds iteration with role separation. **PSRO** [10] provides the game-theoretic framework: each pass adds a candidate to an empirical game.

7 Discussion

Task-dependent advantage. Autoreason is not universally superior. It excels where genuine tradeoffs exist between competing valid perspectives. For tasks with concrete, verifiable requirements, simpler methods suffice. The recommendation: use autoreason when you don't know which framing will produce the best result; use critique-and-revise when the improvement direction is clear.

CoT judges should be the default. The single cheapest improvement: adding "think step by step about each version" to the judge prompt. No architecture changes, 3× faster convergence, more decisive scores.

Convergence as practical stopping rule. The threshold does not guarantee optimality. It filters noise. Post-convergence passes on Task 1 produced no improvement (pass 15 beat pass 25, 6–1). The Monte Carlo results show 20% non-convergence at 30 passes, meaning some runs need a cap. The Elo analysis confirms quality plateaus early regardless of whether the consecutive-wins threshold is met.

Bloat/prune oscillation signals underdetermined scope, and scope turns out to be the central variable in autoreason's effectiveness. Figure 10 shows the structural story: autoreason grows steadily then stabilizes, while baselines either stagnate (conservative) or bloat monotonically (critique-and-revise). With stronger models, this oscillation becomes one-directional drift (Section 4.10)—but scope constraints eliminate it entirely, restoring both convergence and quality.

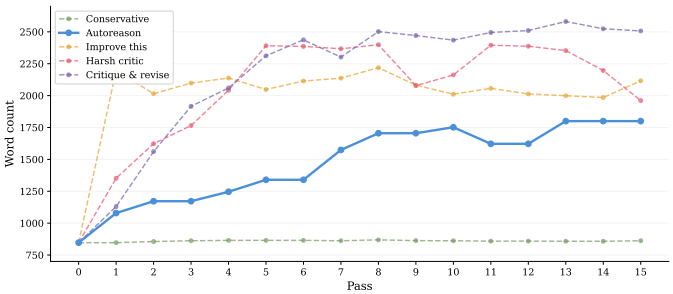

Figure 11. Word count trajectories across 15 passes. All methods start from the same 847-word initial output. Autoreason (solid blue) grows controllably; baselines either stagnate or bloat.

Model scaling is a scope problem. Two rounds of remedy experiments (Section 4.10) systematically eliminated alternative hypotheses. The failure is not about stopping criteria (margin requirement converges but produces mediocre output), evaluation bias (anchored judges don't converge), or synthesis design (subtractive synthesis doesn't converge). It is about scope: on a constrained 500-word pitch task, Sonnet 4.6 converges cleanly in 10 passes with A scoring 9 on the final pass. The synthesis operator needs a bounded improvement space. When the task constrains scope mechanically, the incumbent can reach a ceiling. When it doesn't, there is always something to add.

Limitations. All results from one model family (Anthropic). No human evaluation. No ablation studies isolating individual components. The constrained-task result (Section 4.10) is from a single task and model pair—generalization across diverse constrained tasks and model families remains untested. The paper-writing experiment used a different model (Opus) than the main experiments (Sonnet), limiting direct comparability. The scaling remedy experiments each ran once; Monte Carlo replication would strengthen the convergence claims.

8 Conclusion

Autoreason demonstrates that systematic isolation and blind evaluation can construct useful fitness functions for subjective domains. Across 5 tasks with Sonnet 4, it ranked first or second in every comparison. Chain-of-thought judging accelerates convergence 3×. Retroactive analysis confirms fully transitive preferences and early quality plateau. The method's advantage is clearest on tasks with genuine ambiguity, and its value requires iteration—a single pass does not justify the overhead. However, the Sonnet 4.6 scaling experiments reveal that the method's effectiveness depends on task scope. On scope-ambiguous tasks, stronger models break convergence by producing syntheses that always outcompete the incumbent—and neither evaluation modifications (anchored judges, subtractive synthesis) nor convergence modifications (margin requirement) recover quality, only termination. On scope-constrained tasks, the same stronger model converges cleanly in 10 passes and wins a blind quality comparison against all baselines—including the methods that beat autoreason on the unconstrained version of the task. The synthesis operator needs a bounded improvement space to reach a ceiling. The practical recommendation: pair autoreason with explicit scope constraints, especially as models improve. The broader insight: the obstacle to LLM self-improvement in subjective domains is not capability but the interaction between evaluation methodology and task specification. Both must be right.

References

[1] A. Karpathy. Autoresearch, 2024.

[2] G. Orlanski et al. SlopCodeBench: Benchmarking how coding agents degrade over iterative tasks. *arXiv:2603.24755*, 2026.

[3] J. Zhang et al. ACE: Agentic context engineering. *ICLR*, 2026.

[4] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023. *arXiv:2303.17651*.

[5] J. Huang et al. Large language models cannot self-correct reasoning yet without external feedback. *arXiv:2310.01798*, 2023.

[6] G. Irving, P. Christiano, D. Amodei. AI safety via debate. *arXiv:1805.00899*, 2018.

[7] Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.

[8] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023. *arXiv:2306.05685*.

[9] A. Karpathy. LLM Council, 2025.

[10] M. Lanctot et al. A unified game-theoretic approach to multiagent reinforcement learning. *NeurIPS*, 2017.

[11] K. Arrow. *Social Choice and Individual Values*. Wiley, 1951.

A Prompts

A.1 Critic

```
System: You are a critical reviewer. Your only job is to find real problems. Be specific
and concrete. Do not suggest fixes.
User: Find real problems with this proposal. Focus on: things that won't work as
described, complexity that doesn't pay for itself, assumptions that are wrong, missing
pieces. Do NOT propose fixes. Just the problems.
```

A.2 Author B

```
System: You are a senior consultant revising a proposal based on specific criticisms.
Address each valid criticism directly. Do not make changes not motivated by an identified
problem.
User: [TASK] + [VERSION A] + [CRITIC OUTPUT] Revise to address these problems. For each
change, state which problem it fixes.
```

A.3 Synthesizer

```
System: You are given two versions as equal inputs. Take the strongest elements from each
and produce a coherent synthesis. This is not a compromise.
User: [TASK] + [VERSION X] + [VERSION Y] (labels randomized)
```

A.4 Judge (Baseline)

```
System: You are an independent evaluator. You have no authorship stake in any version.
User: [TASK] + Three proposals (randomized labels and order). Rank best to worst.
RANKING: [best], [second], [worst]
```

A.5 Judge (Chain-of-Thought)

```
System: You are an independent evaluator. Think carefully before deciding.
User: [TASK] + Three proposals. For each, think step by step: 1. What does it get
right? 2. What does it get wrong or miss? 3. Are numbers and claims defensible? 4. Is
detail appropriate or bloated? After reasoning, rank all three. RANKING: [best], [second],
[worst]
```

B Trajectory Trees

Figure 11 shows trajectory trees for all experiments.

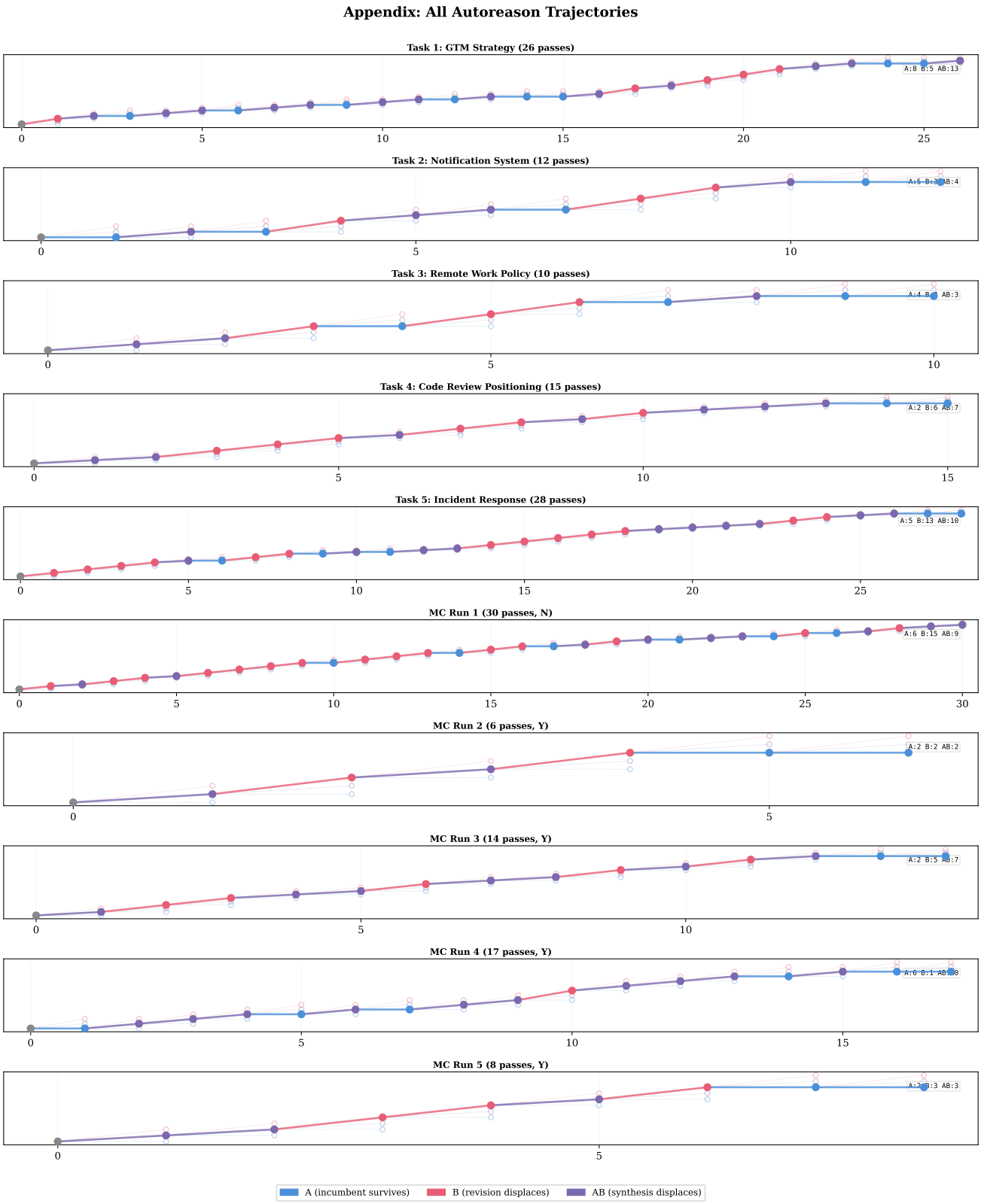

Figure 12. All trajectories. Tasks 2–3 converge quickly (10–12 passes). Task 5 requires 28 passes. Monte Carlo runs show variable convergence speed (6–30 passes) on the same task.